UD4. Manipulación de datos: subconsultas e operacións complexas de edición

Nas sesións anteriores traballamos coas principais sentenzas DML de MariaDB: `INSERT`, `UPDATE` e `DELETE`. Estas sentenzas permiten inserir, modificar e eliminar rexistros dunha táboa, normalmente utilizando valores concretos ou condicións sinxelas.

Porén, en moitas situacións os datos que necesitamos inserir, modificar ou eliminar dependen do contido doutras táboas. Por exemplo, podemos necesitar actualizar o prezo dos produtos pertencentes a unha categoría determinada, inserir nunha táboa os alumnos que cumpren unha condición ou eliminar as matrículas correspondentes a cursos que xa finalizaron.

Para resolver este tipo de operacións utilizaremos subconsultas e combinacións de táboas. Estas ferramentas permiten construír sentenzas de edición máis flexibles e complexas, nas que os resultados dunha consulta determinan os rexistros que se van modificar.

1. Subconsultas en sentenzas de modificación

1.1. Concepto de subconsulta

Unha subconsulta é unha consulta SQL incluída dentro doutra sentenza SQL. A consulta interna execútase para obter información que posteriormente utiliza a consulta externa.

As subconsultas poden utilizarse en diferentes operacións SQL, incluídas as sentenzas de modificación de datos:

- `INSERT`: inserir rexistros obtidos mediante unha consulta.
- `UPDATE`: modificar rexistros utilizando valores ou condicións calculadas por outra consulta.
- `DELETE`: eliminar rexistros en función dos resultados dunha consulta.

A estrutura xeral dunha subconsulta consiste en introducir unha sentenza `SELECT` dentro doutra operación SQL, normalmente entre parénteses.

Exemplo de subconsulta

```
SELECT *
FROM alumnos
WHERE curso_id = (
    SELECT id
    FROM cursos
    WHERE nombre = 'DAW1'
);
```

A consulta interna busca o identificador do curso chamado DAW1. A consulta externa utiliza ese identificador para seleccionar os alumnos que pertencen a ese curso.

1.2. Tipos de subconsultas

Segundo o resultado que devolven, podemos distinguir diferentes tipos de subconsultas:

Tipo	Resultado	Exemplo de uso
Escalar	Un único valor: unha fila e unha columna.	Comparar cun identificador ou cunha media.
De varias filas	Unha lista de valores.	Utilizar os operadores `IN` ou `NOT IN`.
De varias columnas	Un conxunto de columnas.	Comparacións entre varias columnas relacionadas.
Correlacionada	Depende dos valores da consulta externa.	Comprobar a existencia de rexistros relacionados.

Importante: cando unha subconsulta se utiliza cun operador de comparación como `=`, `>` ou `<`, normalmente debe devolver un único valor. Se pode devolver varios resultados, débense utilizar operadores como `IN` ou `EXISTS`.

2. Inserción de datos mediante subconsultas

2.1. INSERT INTO ... SELECT

A sentenza `INSERT INTO ... SELECT` permite inserir nunha táboa os rexistros obtidos mediante unha consulta. A diferenza do `INSERT` tradicional, non é necesario indicar manualmente cada conxunto de valores: os datos proceden doutra táboa ou dunha combinación de táboas.

A súa estrutura xeral é a seguinte:

```
INSERT INTO táboa_destino (columna1, columna2, ...)
SELECT columna1, columna2, ...
FROM táboa_orixe
WHERE condición;
```

A consulta `SELECT` debe devolver o mesmo número de columnas que as indicadas no `INSERT`, e os tipos de datos deben ser compatibles. A orde das columnas tamén debe corresponderse.

2.2. Inserir datos dunha táboa noutra

Supoñamos que queremos crear unha táboa que almacene os alumnos que pertencen a un curso determinado. Primeiro creamos a táboa de destino:

```
CREATE TABLE alumnos_daw1 (
    id INT,
    nombre VARCHAR(100),
    edad INT
) ENGINE=InnoDB;
```

Agora podemos inserir os alumnos que pertencen ao curso DAW1 mediante unha consulta:

```
INSERT INTO alumnos_daw1 (id, nombre, edad)
SELECT id, nombre, edad
FROM alumnos
WHERE curso_id = (
    SELECT id
    FROM cursos
    WHERE nombre = 'DAW1'
);
```

A subconsulta obtén o identificador do curso DAW1. A consulta principal selecciona os alumnos dese curso e o `INSERT` incorpora os resultados á nova táboa.

> **Atención:** se a táboa de destino xa contén os mesmos rexistros e non existe unha restrición que impida a duplicación, a operación pode inserir datos repetidos. É importante comprobar previamente o contido da táboa ou utilizar unha condición que evite duplicados.

2.3. Inserción de datos procedentes de varias táboas

A consulta utilizada nun `INSERT INTO ... SELECT` pode combinar información de varias táboas mediante `JOIN`. Isto permite inserir rexistros que conteñen datos relacionados.

Por exemplo, podemos crear unha táboa que almacene o nome de cada alumno xunto co nome do curso no que está matriculado:

```
CREATE TABLE resumo_alumnos (
    alumno_id INT,
    alumno VARCHAR(100),
    curso VARCHAR(50)
) ENGINE=InnoDB;
```

Para encher esta táboa utilizamos unha combinación de alumnos e cursos:

```
INSERT INTO resumo_alumnos (alumno_id, alumno, curso)
SELECT
    a.id,
    a.nombre,
    c.nombre
FROM alumnos a
INNER JOIN cursos c
    ON a.curso_id = c.id;
```

A consulta selecciona os datos de ambas as táboas e combina os rexistros mediante a relación entre `alumnos.curso_id` e `cursos.id`. O resultado completo insírese en `resumo_alumnos`.

2.4. Inserción de resultados calculados

A consulta que acompaña o `INSERT` tamén pode realizar operacións de cálculo, agrupación e filtrado. Por exemplo, podemos crear unha táboa que almacene o número de alumnos de cada curso:

```
CREATE TABLE resumo_cursos (
    curso VARCHAR(50),
    total_alumnos INT
) ENGINE=InnoDB;

INSERT INTO resumo_cursos (curso, total_alumnos)
SELECT
    c.nombre,
    COUNT(a.id)
FROM cursos c
LEFT JOIN alumnos a
    ON c.id = a.curso_id
GROUP BY c.id, c.nombre;
```

Neste caso, a consulta combina as táboas, agrupa os resultados por curso e calcula o número de alumnos de cada un. O resultado de cada grupo convértese nun rexistro da táboa de destino.

> **Idea clave:** `INSERT INTO ... SELECT` non está limitado a copiar rexistros. Pode inserir resultados de consultas complexas que inclúan combinacións, condicións, funcións de agregación e agrupacións.

3. Actualización de datos mediante subconsultas

3.1. Uso de subconsultas en UPDATE

Unha subconsulta tamén pode proporcionar os valores que se van asignar nunha actualización. Isto resulta especialmente útil cando o novo valor depende doutra táboa.

A estrutura xeral é:

```
UPDATE táboa
SET columna = (
    SELECT valor
    FROM outra_táboa
    WHERE condición
)
WHERE condición;
```

Por exemplo, supoñamos que queremos actualizar o identificador dun curso asociado a un alumno, utilizando o nome do curso para localizar o seu novo identificador:

```
UPDATE alumnos
SET curso_id = (
    SELECT id
    FROM cursos
    WHERE nombre = 'DAM1'
)
WHERE nombre = 'Ana García';
```

A subconsulta obtén o identificador de DAM1. A sentenza principal asigna ese identificador ao alumno Ana García.

> **Importante:** a subconsulta utilizada para asignar un valor debe devolver como máximo unha fila. Se devolve varias filas, MariaDB producirá un erro.

3.2. Actualización condicionada por outra táboa

Tamén podemos utilizar unha subconsulta dentro da condición `WHERE`. Desta maneira, seleccionamos os rexistros que se van actualizar en función da información doutra táboa.

Por exemplo, podemos incrementar a idade dun alumno se pertence ao curso DAW1:

```
UPDATE alumnos
SET edad = edad + 1
WHERE curso_id = (
    SELECT id
    FROM cursos
    WHERE nombre = 'DAW1'
);
```

A subconsulta identifica o curso e a actualización aplícase a todos os alumnos que pertencen a el.

3.3. Actualización mediante valores calculados

As subconsultas tamén poden calcular valores que se utilizarán nunha actualización. Por exemplo, se temos unha táboa de produtos e outra de categorías, poderiamos actualizar o prezo dos produtos utilizando un incremento definido para cada categoría.

O seguinte exemplo supón que existe unha táboa `categorias` cun campo `incremento` e unha táboa `productos` relacionada mediante `categoria_id`:

```
UPDATE productos p
SET precio = precio * (
    1 + (
        SELECT c.incremento
        FROM categorias c
        WHERE c.id = p.categoria_id
    )
)
WHERE EXISTS (
    SELECT 1
    FROM categorias c
    WHERE c.id = p.categoria_id
);
```

Para cada produto, a subconsulta obtén o incremento correspondente á súa categoría. O prezo actualízase aplicando ese incremento.

Este exemplo utiliza unha subconsulta correlacionada, xa que fai referencia á columna `p.categoria_id` da consulta externa.

4. Eliminación de datos mediante subconsultas

4.1. Uso de subconsultas en DELETE

A sentenza `DELETE` permite utilizar subconsultas para determinar que rexistros deben eliminarse. Desta maneira, a condición de eliminación pode depender dos datos doutras táboas.

A estrutura xeral é:

```
DELETE FROM táboa
WHERE columna IN (
    SELECT columna
    FROM outra_táboa
    WHERE condición
);
```

Por exemplo, podemos eliminar todos os alumnos que pertencen ao curso DAW1:

```
DELETE FROM alumnos
WHERE curso_id = (
    SELECT id
    FROM cursos
    WHERE nombre = 'DAW1'
);
```

A subconsulta obtén o identificador do curso DAW1 e a sentenza `DELETE` elimina os alumnos asociados a ese curso.

4.2. Eliminar rexistros en función dunha consulta

Outro exemplo sería eliminar as matrículas de alumnos que pertencen a un curso determinado:

```
DELETE FROM matriculas
WHERE alumno_id IN (
    SELECT id
    FROM alumnos
    WHERE curso_id = (
        SELECT id
        FROM cursos
        WHERE nombre = 'DAW1'
    )
);
```

Neste caso hai dúas subconsultas. A máis interna localiza o curso DAW1; a intermedia obtén os identificadores dos alumnos dese curso; e a sentenza principal elimina as matrículas correspondentes a eses alumnos.

> **Precaución:** antes de executar un `DELETE` con subconsultas, é recomendable executar a consulta `SELECT` equivalente para comprobar exactamente que rexistros serán afectados.

5. Operadores IN, NOT IN, EXISTS e NOT EXISTS

Cando unha subconsulta pode devolver varios rexistros, necesitamos operadores que permitan comprobar se un valor está incluído nos resultados ou se existe algún rexistro que cumpra unha condición.

5.1. Operador IN

O operador `IN` comproba se un valor coincide con algún dos valores devoltos por unha subconsulta.

Por exemplo, eliminar as matrículas dos alumnos que pertencen a DAW1 ou DAM1:

```
DELETE FROM matriculas
WHERE alumno_id IN (
    SELECT id
    FROM alumnos
    WHERE curso_id IN (
        SELECT id
        FROM cursos
        WHERE nombre IN ('DAW1', 'DAM1')
    )
);
```

A subconsulta devolve os identificadores dos alumnos dos dous cursos. A operación elimina as matrículas cuxo alumno aparece nesa lista.

5.2. Operador NOT IN

O operador `NOT IN` realiza a comprobación contraria: selecciona os rexistros cuxo valor non aparece entre os resultados da subconsulta.

Por exemplo, eliminar os alumnos que non pertencen a ningún curso:

```
DELETE FROM alumnos
WHERE curso_id NOT IN (
    SELECT id
    FROM cursos
);
```

Esta sentenza elimina os alumnos cuxo identificador de curso non aparece na táboa `cursos`.

> **Atención co valor NULL:** se unha subconsulta utilizada con `NOT IN` devolve algún valor `NULL`, o resultado das comparacións pode non ser o esperado. Cando existe esta posibilidade, adoita ser máis seguro utilizar `NOT EXISTS`.

5.3. Operador EXISTS

O operador `EXISTS` comproba se unha subconsulta devolve polo menos un rexistro. Non importa o valor concreto das columnas seleccionadas: unicamente interesa saber se existe algún resultado.

Por exemplo, podemos actualizar a idade dos alumnos que teñen polo menos unha matrícula:

```
UPDATE alumnos a
SET edad = edad + 1
WHERE EXISTS (
    SELECT 1
    FROM matriculas m
    WHERE m.alumno_id = a.id
);
```

A subconsulta busca matrículas relacionadas con cada alumno. Se atopa polo menos unha, `EXISTS` devolve verdadeiro e o alumno actualízase.

5.4. Operador NOT EXISTS

O operador `NOT EXISTS` comproba que unha subconsulta non devolve ningún rexistro. Utilízase para localizar elementos que non teñen rexistros relacionados.

Por exemplo, eliminar os alumnos que non teñen ningunha matrícula:

```
DELETE FROM alumnos a
WHERE NOT EXISTS (
    SELECT 1
    FROM matriculas m
    WHERE m.alumno_id = a.id
);
```

Para cada alumno, a subconsulta comproba se existe algunha matrícula asociada. Se non existe ningunha, o alumno elimínase.

5.5. Comparación dos operadores

Operador	Función	Uso habitual
`IN`	Comproba se un valor pertence a unha lista de resultados.	Modificar rexistros relacionados con un conxunto de valores.
`NOT IN`	Comproba se un valor non pertence á lista.	Localizar valores que non aparecen noutra consulta.
`EXISTS`	Comproba se existe polo menos un resultado.	Actuar cando hai rexistros relacionados.
`NOT EXISTS`	Comproba que non existe ningún resultado.	Localizar rexistros sen elementos relacionados.

6. Operacións de edición con varias táboas

6.1. UPDATE con JOIN

MariaDB permite utilizar combinacións de táboas directamente nunha sentenza `UPDATE`. Esta opción facilita a modificación de rexistros cando o novo valor ou a condición depende doutra táboa.

A estrutura xeral é:

```
UPDATE táboa1
JOIN táboa2
    ON táboa1.campo = táboa2.campo
SET táboa1.columna = valor
WHERE condición;
```

Por exemplo, queremos actualizar a idade dos alumnos que pertencen ao curso DAW1:

```
UPDATE alumnos a
INNER JOIN cursos c
    ON a.curso_id = c.id
SET a.edad = a.edad + 1
WHERE c.nombre = 'DAW1';
```

A combinación relaciona alumnos e cursos. A condición selecciona o curso DAW1 e a actualización modifica a idade de todos os alumnos dese curso.

6.2. Actualizar información utilizando varias táboas

Podemos combinar máis de dúas táboas nunha mesma operación. Por exemplo, imaxinemos que queremos modificar a nota das matrículas dos alumnos que pertencen a DAW1.

```
UPDATE matriculas m
INNER JOIN alumnos a
    ON m.alumno_id = a.id
INNER JOIN cursos c
    ON a.curso_id = c.id
SET m.nota = 5
WHERE c.nombre = 'DAW1';
```

A sentenza combina as tres táboas para localizar as matrículas dos alumnos do curso indicado. Só se modifican os rexistros da táboa `matriculas`.

> **Importante:** nun `UPDATE` con varias táboas, hai que indicar claramente que táboa se quere modificar. As combinacións permiten localizar os rexistros relacionados, pero non implican que todas as táboas combinadas se actualicen automaticamente.

7. Eliminación de rexistros mediante combinacións de táboas

7.1. DELETE con JOIN

Do mesmo xeito que podemos actualizar información mediante combinacións, tamén podemos eliminar rexistros utilizando condicións baseadas noutras táboas.

MariaDB permite indicar que táboa ou táboas se van eliminar nun `DELETE` con `JOIN`.

A estrutura básica para eliminar rexistros dunha táboa é:

```
DELETE alias
FROM táboa1 alias
JOIN táboa2
    ON alias.campo = táboa2.campo
WHERE condición;
```

Por exemplo, podemos eliminar as matrículas dos alumnos que pertencen ao curso DAW1:

```
DELETE m
FROM matriculas m
INNER JOIN alumnos a
    ON m.alumno_id = a.id
INNER JOIN cursos c
    ON a.curso_id = c.id
WHERE c.nombre = 'DAW1';
```

A sentenza combina as tres táboas para localizar as matrículas dos alumnos de DAW1. A cláusula `DELETE m` indica que unicamente se eliminan rexistros da táboa `matriculas`.

7.2. Eliminar rexistros de varias táboas

MariaDB tamén permite eliminar rexistros de varias táboas relacionadas nunha mesma sentenza, sempre que se indique que táboas son as afectadas.

Por exemplo:

```
DELETE m, a
FROM matriculas m
INNER JOIN alumnos a
    ON m.alumno_id = a.id
WHERE a.nombre = 'Ana García';
```

Esta operación elimina tanto as matrículas de Ana García como o seu rexistro na táboa `alumnos`.

> **Precaución:** eliminar rexistros de varias táboas require especial atención ás relacións de clave foránea. Se existen outras táboas que referencian os rexistros que se queren eliminar, as restricións de integridade poden impedir a operación. É necesario comprobar previamente as dependencias e as políticas de eliminación definidas nas claves foráneas.

7.3. DELETE con LEFT JOIN

Unha combinación `LEFT JOIN` tamén permite localizar rexistros que non teñen correspondencia noutra táboa. É unha técnica útil para identificar e eliminar rexistros sen relacións.

Por exemplo, podemos eliminar alumnos que non teñen ningunha matrícula:

```
DELETE a
FROM alumnos a
LEFT JOIN matriculas m
    ON a.id = m.alumno_id
WHERE m.alumno_id IS NULL;
```

O `LEFT JOIN` conserva todos os alumnos, aínda que non teñan matrículas. Cando non existe unha matrícula asociada, as columnas da táboa `matriculas` aparecen como `NULL`. A condición `WHERE m.alumno_id IS NULL` identifica precisamente eses alumnos.

8. Resolución de operacións complexas de edición

8.1. Combinar subconsultas e condicións

As operacións de edición máis complexas adoitan combinar diferentes técnicas:

- Subconsultas para obter identificadores ou valores.
- Operadores `IN` e `EXISTS` para comprobar relacións.
- Combinacións `JOIN` para acceder a información de varias táboas.
- Condicións `WHERE` para limitar os rexistros afectados.
- Funcións de agregación para calcular valores a partir de varios rexistros.

Antes de executar unha operación complexa é conveniente separar o problema en pasos e comprobar os resultados das consultas que determinan os rexistros afectados.

8.2. Exemplo: actualizar notas segundo a media do curso

Supoñamos que queremos incrementar nun punto as notas dos alumnos cuxa cualificación está por baixo da media do seu curso. Para iso necesitamos relacionar matrículas, alumnos e cursos, e calcular a media das notas de cada curso.

```
UPDATE matriculas m
INNER JOIN alumnos a
    ON m.alumno_id = a.id
INNER JOIN cursos c
    ON a.curso_id = c.id
SET m.nota = m.nota + 1
WHERE m.nota < (
    SELECT AVG(m2.nota)
    FROM matriculas m2
    INNER JOIN alumnos a2
        ON m2.alumno_id = a2.id
    WHERE a2.curso_id = c.id
);
```

A consulta interna calcula a media das notas dos alumnos do mesmo curso que o rexistro que se está avaliando. A actualización incrementa un punto as notas inferiores a esa media.

Trátase dunha subconsulta correlacionada porque utiliza o identificador do curso da consulta externa.

8.3. Exemplo: inserir alumnos que aínda non teñen matrícula

Outra situación habitual consiste en inserir nunha táboa os alumnos que aínda non teñen ningún rexistro asociado noutra táboa.

Supoñamos que existe unha táboa `alumnos_sen_matricula` coas columnas `alumno_id` e `nombre`. Podemos enchela mediante a seguinte operación:

```
INSERT INTO alumnos_sen_matricula (alumno_id, nombre)
SELECT a.id, a.nombre
FROM alumnos a
WHERE NOT EXISTS (
    SELECT 1
    FROM matriculas m
    WHERE m.alumno_id = a.id
);
```

A consulta selecciona os alumnos para os que non existe ningunha matrícula. Os resultados insírense directamente na táboa de destino.

8.4. Exemplo: eliminar cursos sen alumnos

Podemos utilizar `NOT EXISTS` para localizar cursos que non teñen ningún alumno asociado:

```
SELECT *
FROM cursos c
WHERE NOT EXISTS (
    SELECT 1
    FROM alumnos a
    WHERE a.curso_id = c.id
);
```

Unha vez comprobados os resultados, poderiamos eliminar eses cursos:

```
DELETE FROM cursos c
WHERE NOT EXISTS (
    SELECT 1
    FROM alumnos a
    WHERE a.curso_id = c.id
);
```

A eliminación só será posible se non existen outras restricións ou dependencias que impidan borrar eses cursos.

9. Boas prácticas nas operacións complexas

As operacións DML que inclúen subconsultas e combinacións poden afectar a un número elevado de rexistros. Unha condición incorrecta pode provocar modificacións ou eliminacións non desexadas.

Para reducir este risco é recomendable seguir estas pautas:

1. **Analizar as relacións:** identificar as táboas implicadas e as claves que permiten relacionalas.
2. **Construír primeiro a consulta:** desenvolver un `SELECT` que devolva exactamente os rexistros que se queren modificar.
3. **Comprobar os resultados:** revisar os identificadores e os datos seleccionados antes de executar a operación DML.
4. **Utilizar condicións específicas:** evitar actualizacións ou eliminacións sen unha cláusula `WHERE` cando non se pretende afectar a toda a táboa.
5. **Respectar a integridade referencial:** comprobar as claves foráneas e as políticas `ON DELETE` e `ON UPDATE`.
6. **Utilizar transaccións:** cando as operacións o permitan, executar os cambios dentro dunha transacción para poder revisalos antes de confirmalos.

9.1. Uso de transaccións para protexer operacións DML

As táboas InnoDB permiten executar operacións DML dentro dunha transacción. Isto é especialmente útil cando unha modificación implica varias sentenzas que deben completarse correctamente.

Por exemplo:

```
START TRANSACTION;

UPDATE alumnos
SET edad = edad + 1
WHERE curso_id = 1;

DELETE FROM matriculas
WHERE alumno_id IN (
    SELECT id
    FROM alumnos
    WHERE curso_id = 1
);

-- Se todo é correcto:
COMMIT;

-- Se se detecta algún problema, antes do COMMIT:
-- ROLLBACK;
```

A sentenza `COMMIT` confirma os cambios realizados. Pola contra, `ROLLBACK` desfai os cambios da transacción que aínda non foron confirmados.

> **Importante:** as transaccións protexen as operacións DML compatibles con InnoDB, pero non permiten desfacer automaticamente operacións DDL como `CREATE TABLE`, `DROP TABLE` ou `TRUNCATE TABLE`.

10. Resumo das principais estruturas

Operación	Estrutura	Finalidade
INSERT con consulta	`INSERT INTO ... SELECT`	Inserir resultados dunha consulta.
UPDATE con subconsulta	`UPDATE ... SET ... (SELECT ...)`	Utilizar resultados doutra consulta para actualizar valores.
DELETE con subconsulta	`DELETE ... WHERE ... IN (SELECT ...)`	Eliminar rexistros identificados mediante unha consulta.
Comprobar pertenza	`IN` / `NOT IN`	Comprobar se un valor está ou non nun conxunto de resultados.
Comprobar existencia	`EXISTS` / `NOT EXISTS`	Comprobar se existen rexistros relacionados.
Actualizar con combinacións	`UPDATE ... JOIN`	Modificar rexistros utilizando información doutras táboas.
Eliminar con combinacións	`DELETE ... JOIN`	Eliminar rexistros utilizando condicións baseadas noutras táboas.

11. Guión de comprobación

Antes de executar unha operación complexa de inserción, actualización ou eliminación, convén comprobar os seguintes aspectos:

1. Identificar a táboa ou táboas que interveñen na operación.
2. Localizar as relacións entre elas mediante as claves primarias e foráneas.
3. Construír a consulta `SELECT` que identifica os rexistros afectados.
4. Comprobar que a consulta devolve os resultados esperados.
5. Seleccionar a operación adecuada: `INSERT`, `UPDATE` ou `DELETE`.
6. Decidir se é necesario utilizar unha subconsulta ou unha combinación `JOIN`.
7. Revisar as condicións e comprobar que non se afectan rexistros adicionais.
8. Comprobar as restricións de integridade referencial.
9. Executar a operación e verificar os cambios realizados.
10. Se se utiliza unha transacción, confirmar os cambios con `COMMIT` ou desfacelos con `ROLLBACK`.

Conclusión: as subconsultas e as combinacións de táboas permiten realizar operacións de edición baseadas na información existente na base de datos. O seu uso facilita a automatización de tarefas complexas e evita ter que consultar manualmente identificadores ou valores para utilizalos posteriormente noutras sentenzas SQL.